

DEVELOPING A DJANGO-BASED DATA INSIGHTS PLATFORM FOR ENERGY FORECASTING

Maria Fernanda Cladera Melgar

University of Lausanne

MariaFernanda.CladeraMelgar@unil.ch

Abstract – This project presents the development of a comprehensive data insights platform for energy consumption forecasting, addressing the growing need for accurate energy management and sustainability. By integrating machine learning techniques with data preprocessing and management, the platform effectively predicts energy consumption for different zones. Using the Django framework ensures scalability and maintainability, while the XGBoost model provides high forecasting accuracy. The platform features a user-friendly interface with interactive data visualization tools, enhancing decision-making processes for energy suppliers. This project contributes to improved operational efficiency, resource management, and environmental sustainability.

Keywords: *Energy Consumption Forecasting, Data Insights Platform, Machine Learning, XGBoost, Django, Data Visualization, Energy Management, Sustainability.*

I. INTRODUCTION

Energy forecasting is a crucial element in the pursuit of energy efficiency and sustainability, especially for energy suppliers who need to balance supply and demand accurately. As urbanization continues to accelerate globally, the demand for energy has seen a significant increase. Energy suppliers face the challenge of predicting energy consumption to ensure the stability and reliability of the power grid while

optimizing resource allocation and minimizing costs [1].

The need to optimize energy usage is imperative not only for economic reasons but also for environmental sustainability.

Accurate energy forecasting models are essential for effective energy management by suppliers. These models help in planning energy resources, balancing supply and demand, and ensuring the stability and reliability of the power grid [2]. Various approaches have been developed for energy consumption modeling and forecasting, including physics-based methods, data-driven techniques, and hybrid models [3].

Data-driven methods, in particular, have emerged as highly effective due to their ability to handle large amounts of energy-related data generated by smart sensors and meters. However, the implementation of these models poses challenges due to the nonlinear, dynamic, and complex nature of energy consumption data, which often exhibit trends, seasonality, and irregular patterns influenced by various exogenous factors such as weather conditions and consumer behavior [4]. Therefore, a comprehensive data insights platform that integrates advanced analytics tools is needed to enhance the accuracy and reliability of energy consumption forecasts for energy suppliers.

A robust data insights platform must address several key challenges. These include data collection and integration from various sources, handling and preprocessing raw data to ensure its quality and consistency and deploying advanced analytical tools to extract meaningful

insights. Effective data visualization and reporting mechanisms are also essential to facilitate decision-making processes. The complexity of raw energy data necessitates sophisticated preprocessing techniques to handle missing values, anomalies, and noise, which are prevalent in real-world datasets [3].

To effectively tackle these challenges, the selection of appropriate web development technologies becomes crucial. The right choice of technology can significantly impact the efficiency, scalability, and maintainability of the platform. By leveraging modern web frameworks, it is possible to create a robust infrastructure capable of handling the complexities of energy data analytics while providing an intuitive user interface.

A. Web development Technologies

There are numerous technologies available for the development of web applications, including Angular, React, Vue.js, Laravel, and Ruby on Rails. However, some of the most popular frameworks for building comprehensive data insights platforms are Flask, Streamlit, Dash, and Django. Each of these frameworks offers unique features and benefits, catering to different needs and preferences in web development.

- Flask: A lightweight WSGI web application framework in Python, designed with simplicity and flexibility, making it easy to use and extend. Suitable for small to medium-sized applications.
- Streamlit: An open-source app framework for creating data-driven web applications in Python, ideal for prototyping and deploying machine learning models and data visualizations quickly.
- Dash: A productive Python framework for building web applications, particularly well-suited for creating interactive data visualizations. Ideal for

data scientists wanting to create dashboards using pure Python.

- Django: A high-level Python web framework encouraging rapid development and clean, pragmatic design. It includes built-in features like authentication, database management, and an admin interface, making it a comprehensive solution for building scalable web applications.

Given the need for a comprehensive and scalable analytics solution for energy forecasting, Django emerges as an ideal choice due to its extensive features and ability to handle complex applications. Django's robust framework and built-in tools facilitate efficient data management, complex analytical processing, and intuitive user interface development.

The objective of this project is to create a small-scale analytic software solution for energy consumption data analytics using Django. The proposed platform aims to manage and analyze large-scale energy data efficiently, providing valuable insights for energy suppliers.

II. RESEARCH QUESTIONS

Given the challenges associated with energy consumption forecasting such as handling large volumes of complex, nonlinear, and dynamic data—there is a critical need for a comprehensive data insights platform. This platform will integrate analytics tools to enhance the accuracy and reliability of energy consumption forecasts. Traditional methods often fall short in dealing with the intricacies of modern energy data, necessitating a robust and scalable solution that can handle data collection, preprocessing, visualization, and machine learning model deployment.

Framework	Description	Pros	Cons
Flask	A lightweight WSGI web application framework in Python.	A lightweight WSGI web application framework in Python.	A lightweight WSGI web application framework in Python.
Streamlit	An open-source app framework for creating data-driven web applications in Python.	An open-source app framework for creating data-driven web applications in Python.	An open-source app framework for creating data-driven web applications in Python.
Dash	A productive Python framework for building web applications, especially interactive visualizations.	A productive Python framework for building web applications, especially interactive visualizations.	A productive Python framework for building web applications, especially interactive visualizations.
Django	A high-level Python web framework that encourages rapid development and clean design.	A high-level Python web framework that encourages rapid development and clean design.	A high-level Python web framework that encourages rapid development and clean design.

Table 1. Summary of Webdevelopment Technologie

A. Objectives

To develop a comprehensive platform that provides energy suppliers with advanced analytics and forecasting capabilities for energy consumption data, thereby improving resource management, operational efficiency, and decision-making processes.

Specific objectives:

- Develop a Comprehensive Data Management System.
- Create a User-Friendly Interface for Authentication and Interaction
- Build Data Visualization Tools
- Integrate Machine Learning Models for Energy Forecasting

III. METHODOLOGY

The development of a platform for energy consumption analytics involved identifying and integrating selected data sources, preprocessing data, developing models, and implementing the platform.

A. Data Collection and Integration

1. Data Source Selection and collection

The dataset used in this study is sourced from the UCI Machine Learning Repository [5]. This dataset is related to the power consumption of three different distribution networks in Tetuán City, located in northern Morocco. It is a multivariate, time-series dataset with applications in social science and is typically used for regression tasks.

The dataset contains power consumption data recorded at regular intervals. The primary features include temperature, humidity, Wind Speed, General Diffuse

Flows, Diffuse Flows and the power consumption of 3 zones (Zone 1, Zone 2 and Zone 3) (Fig. 1).

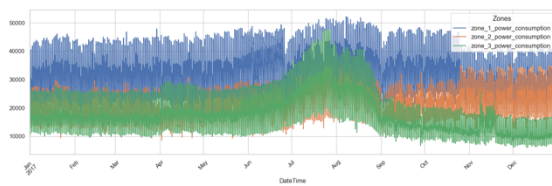


Fig. 1 Data trend - Energy Consumption

The dataset was acquired through direct download from the UCI repository as a *csv* document and save in '*static/rawdata*' file as '*Tetuan_City_power_consumption*'.

B. Data Preprocessing

1. Data Cleaning

The data cleaning process involved loading the raw data from the database into a Pandas DataFrame. Missing values were handled by imputing the mean for numerical columns, excluding the date column. Numerical features were normalized using the MinMaxScaler from scikit-learn to ensure consistent scaling. Finally, the cleaned data was saved back into the SQLite database.

2. Data Transformation

The data transformation process involved creating additional temporal features from the date column, such as hour, day of the week, quarter, month, year, day of the year, and day. Lag features, rolling means, and exponential moving averages were generated for power consumption data to capture trends and patterns over time. These transformations enable more effective analysis and forecasting.

C. Model Development

In another ADA project, the performance of three machine learning models—XGBoost, Random Forest, and LSTM—was evaluated for forecasting energy consumption across three zones in Tetuan

City. These models were chosen for their ability to handle time series data and their proven effectiveness in similar forecasting tasks. XGBoost was selected due to its superior performance in terms of accuracy and computational efficiency.

The dataset was divided into training and test sets, with a split ratio of 70% training data and 30% test data. The XGBoost model was initially trained on the training set. Hyperparameter tuning was performed using random search to optimize the model's performance. Cross-validation with TimeSeriesSplit was employed to evaluate the model, and the Root Mean Squared Error (RMSE) was used as the evaluation metric.

The final model was chosen based on its ability to minimize the RMSE on the test set, demonstrating its effectiveness in forecasting energy consumption across different zones.

D. Platform Development

1. Backend Development

The backend infrastructure was developed using Django. This setup facilitated the server configuration, database connections, and creation of RESTful APIs for data access and model predictions (Fig. 2).

The project is structured into four main applications: *app*, *data*, *ml*, and *users*:

- *App: app*: Manages main views and user interactions, including home, dashboard, analysis, and report pages.
- *App: data*: Handles data management and processing, including models for storing energy consumption data. Implements data cleaning and preprocessing via management commands to ensure data quality and consistency.

- *App: ml*: Integrates machine learning models, particularly XGBoost, for energy consumption forecasting.
- *App: users*: Manages user authentication and authorization, with templates for login and registration.

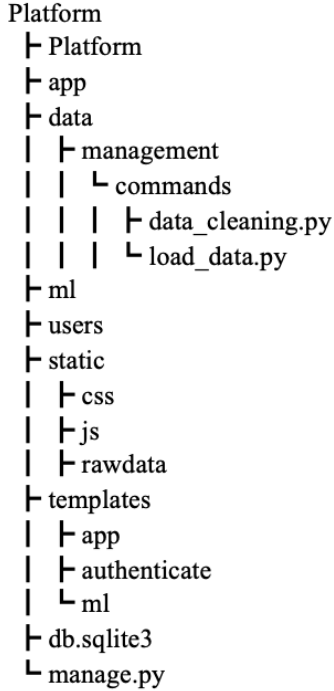


Fig. 2 Tree diagramn backend structure

2. Frontend Development

A user-friendly interface was developed using HTML, CSS, JavaScript, and the React framework. This interface allows users to interact with the platform, visualize data, and access forecasts easily. The design focused on usability and accessibility, ensuring a seamless user experience (Fig. 3).

3. Data Visualization

Interactive and dynamic charts were created using data visualization libraries such as Plotly. These visualizations help users understand energy consumption patterns and statistics, providing valuable insights for decision-making.

4. User Authentication and Authorization

An authentication and authorization mechanisms were implemented to ensure secure access to the platform. Django's built-in authentication system was enhanced with OAuth and JWT for added security, protecting user data and platform integrity.

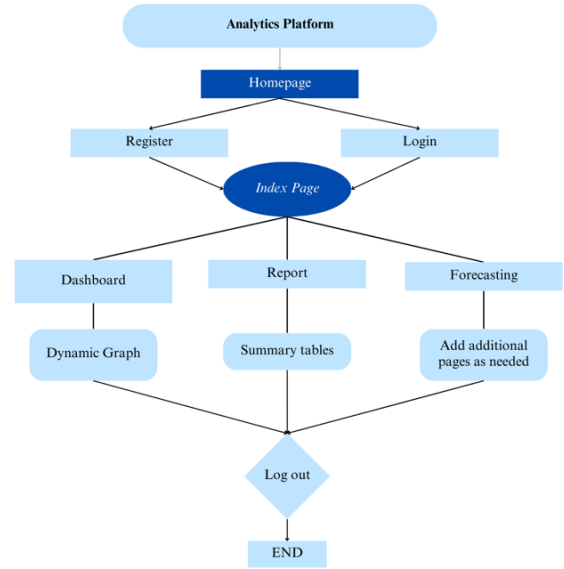


Fig. 3 Frontend Flow

IV. IMPLEMENTATION

Since the objective is to program a small-scale software for businesses, four applications were created (as presented in the methodology section). The goal is that each application is separate so that in the future the scalability of the software can be implemented.

To visualize the software, it is necessary to be inside the Python virtual environment 'venv' and within the project directory 'ADA_AP_FINALPROJECT/Platform'.

- Application: *app*

This application contains most of the visuals of the software, among them the *home.html*, *index.html*, *report.html* and *dashboard.html*.

The two most important python scripts of the 'app' are 'app/views.py' and 'app/urls.py'. In 'app/views.py', the functions for rendering the templates and visualization data were developed, the most important function is 'Dashboard' which generates all the dynamic graphs of trends and histograms.

For the other side, 'app/urls.py' file maps URL paths to view functions, allowing the Django application to handle specific URLs and direct user requests to the appropriate view functions for the correct response. This file is essential because it defines the routing logic, enabling the application to navigate to different views based on the URL requested, thus ensuring proper handling of user interactions and displaying the correct web pages.

navbar and sidebar, and charts were generated with Chart.js in JavaScript.

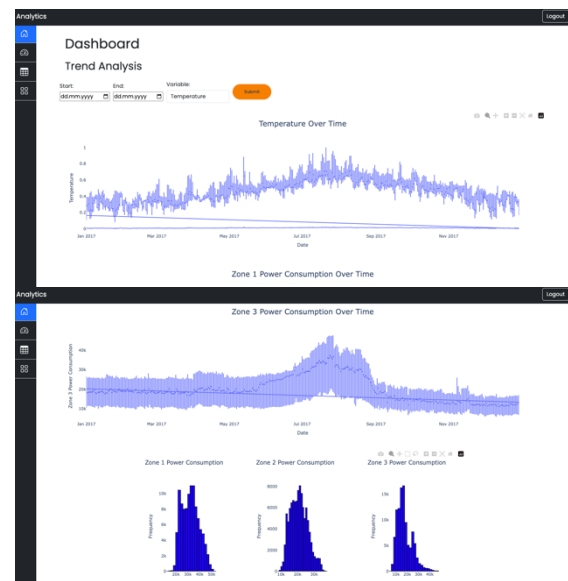


Fig. 7 Dashboard view

• Application: users

The primary function of this app is to handle user authentication and registration. This includes allowing users to log in, log out, and create new accounts. The app integrates with Django's built-in authentication system to manage these functionalities securely and efficiently.

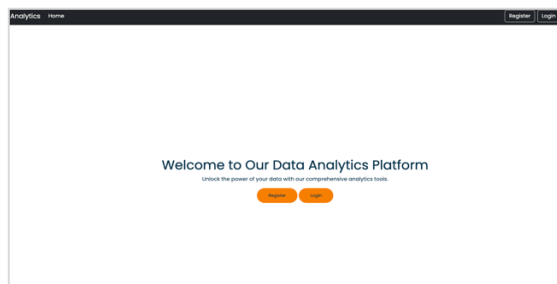


Fig. 4 Homepage view

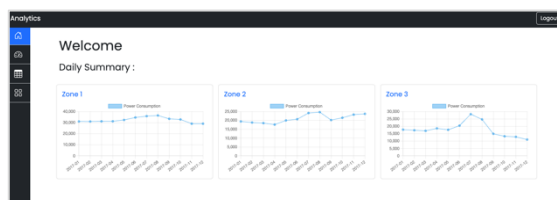


Fig. 5 Index page view

The report page view features a dark header with 'Analytics' and 'Logout' links. The main content area is titled 'Report of Energy Consumption' and includes a table with columns for 'Month', 'Zone 1 Power Consumption', 'Zone 2 Power Consumption', and 'Zone 3 Power Consumption'. The table displays data for months from 2017-01 to 2017-12. A search bar and a 'Showing 1 to 10 of 100 entries' indicator are also present.

Month	Zone 1 Power Consumption	Zone 2 Power Consumption	Zone 3 Power Consumption
2017-01	30226.48	10226.48	17168.00
2017-02	30980.76	16787.78	17339.00
2017-03	30980.77	16787.78	16947.69
2017-04	30980.77	17633.97	16963.17
2017-05	32398.01	19977.29	17823.00
2017-06	34029.54	20570.80	20420.34
2017-07	34029.54	24451.84	28816.10
2017-08	34430.19	24058.22	24648.89
2017-09	33396.68	20980.43	14922.80
2017-10	32827.66	21458.99	12064.01
2017-11	29002.01	23240.46	12862.50
2017-12	29024.17	23680.86	10244.81

Fig. 6 Report page view

Since one of the goals was to create a user-friendly interface, the front-end was developed using HTML and CSS. Bootstrap templates were used for the

The registration and login view features a dark header with 'Analytics Home' and 'Register | Login' links. The main content area is divided into two sections: 'Register' and 'Login'. The 'Register' section includes a form with fields for 'Username' (required, 100 characters or fewer, letters, digits and @/./_/- only), 'Password', and 'Password confirmation' (must be the same as the password above). The 'Login' section includes a form with fields for 'Username' and 'Password'.

Fig. 8 Registration and log in view

Also in the 'users' application, the scripts 'users/views.py' and 'users/urls.py' are present. In 'users/views.py', all the functions related to registration, login, and logout are developed. For registration and login, a POST request is used to submit form data to the server for processing. Specifically, the `register_user` function handles user registration by creating a new user and

logging them in, while the *login_user* function manages user authentication and login.

In '*users/urls.py*', all the URL patterns are defined to map to these views. This includes the paths for login (*/login*), logout (*/logout*), and registration (*/register*), allowing users to navigate to these functionalities within the application.

- Application: data

The primary function is to manage and serve power consumption data. This includes storing the data in the database, cleaning and preprocessing it, and providing an API endpoint for retrieving the data in JSON format.

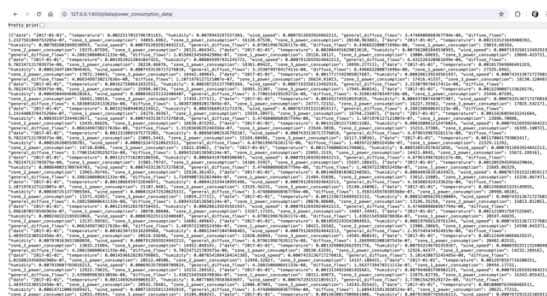


Fig. 9 View of Dataset Endpoint in Json format

In this app, '*data/views.py*' includes the '*power_consumption_data*' function to provide an API endpoint that returns the power consumption dataset in JSON format. In '*data/urls.py*', the URL patterns for the application are also defined.

For loading and cleaning the dataset, there are additional main documents: '*models.py*' defines the structure of the power consumption data in the database, '*data_cleaning.py*' provides a custom Django management command for cleaning and preprocessing the dataset, and '*load_data.py*' offers another custom command for loading power consumption data from a CSV file into the database.

- Application: ml

This application is essential for forecasting task. It uses a XGBoost pre-trained machine learning model to forecast power consumption. In this app we generate the '*forecast_next_month*' view with the monthly predictions based on historical data and displays them in a user-friendly table format.

Date	Zone 1 Power Consumption	Zone 2 Power Consumption	Zone 3 Power Consumption
2018-01-01	26728.922	20449.862	17483.402
2018-01-02	26728.402	20564.893	17487.344
2018-01-03	26738.27	20880.648	17488.072
2018-01-04	26758.272	21000.65	17503.058
2018-01-05	26749.395	21042.219	17529.443
2018-01-06	26764.508	21022.817	17525.895
2018-01-07	26887.906	21022.523	17508.984
2018-01-08	26742.162	21086.817	17488.688
2018-01-09	26752.162	21056.58	17489.375
2018-01-10	26750.344	21071.826	17489.81
2018-01-11	26750.227	21056.102	17517.289
2018-01-12	26760.877	21041.195	17523.213
2018-01-13	26765.102	21022.584	17529.81

Fig. 10 View of Forecast of next month using XGBoost model

V. RESULTS

The developed data insights platform effectively addresses the need for accurate energy consumption forecasting. By integrating machine learning models with robust data preprocessing and management, the platform presents the forecast for energy consumption across three zones in Tetuán City. The use of Django for backend development ensures a scalable and maintainable architecture, capable of handling large volumes of complex data efficiently.

In the initial data preprocessing stage, the cleaning and transformation processes effectively handled missing values and normalized the dataset, ensuring consistency and reliability. The addition of temporal features and lag variables significantly enhanced the predictive power of the machine learning models. The chosen XGBoost model outperformed other evaluated models, including Random Forest and LSTM, in terms of both accuracy and computational efficiency. The final tuned XGBoost model demonstrated a low Root Mean Squared Error (RMSE) on the test set, validating its effectiveness in forecasting energy consumption.

The platform's frontend, developed using HTML, CSS, JavaScript, and React, provides a user-friendly interface that facilitates easy interaction and visualization of data. The use of interactive charts and dynamic dashboards enables users to quickly understand energy consumption patterns and make informed decisions. The robust authentication and authorization mechanisms ensure secure access to the platform, protecting user data and maintaining the integrity of the system.

Overall, the data insights platform integrates comprehensive data management, advanced analytics, and user-centric design to deliver valuable forecasting capabilities.

VI. CONCLUSION

The data insights platform demonstrates the critical role of advanced analytics and machine learning in energy consumption forecasting. Integrating large-scale energy data with effective preprocessing and a powerful predictive model, the platform revolutionizes energy management for suppliers. Using Django, the platform is scalable and maintains high forecasting accuracy through the successful application of the XGBoost model and comprehensive data cleaning.

In conclusion, the platform provides a comprehensive solution for energy consumption forecasting, enhancing decision-making for energy suppliers with a user-friendly interface and interactive data visualization tools.

VII. REFERENCES

[1] M. Bourdeau, X. q. Zhai, E. Nefzaoui, X. Guo and P. Chatellier, "Modeling and forecasting building energy

consumption: A review of data-driven techniques," *Sustainable Cities and Society*, vol. 48, 2019.

- [2] L. Ruiz, R. Rueda, M. Cuéllar and M. Pegalajar, "Energy consumption forecasting based on Elman neural networks with evolutive optimization," *Expert Systems with Applications*, vol. 92, pp. 380-389, 2018.
- [3] E. Mocanu, P. H. Nguyen, M. Gibescu and W. L. Kling, "Deep learning for estimating building energy consumption," *Sustainable Energy, Grids and Networks*, vol. 6, pp. 91-99, 2016.
- [4] R. M. R. Gauthama, S. Nivethitha, M. R. Raman and R. Krithi, "A deep learning framework for building energy consumption forecast," *Renewable and Sustainable Energy Reviews*, vol. 137, 2021.
- [5] A. Salam and A. El Hibaoui, *Power Consumption of Tetouan City*, UCI Machine Learning Repository, 2023.
- [6] A. Salam and A. El Hibaoui, *Power Consumption of Tetouan City*, UCI Machine Learning Repository, 2019.

ChatGTP